

Building a Simple Web Application with PostgreSQL, Node.js, HTML, and JavaScript

Content:

- Overview of the lab
- What topics we will cover:
 - Setting up the environment
 - Creating a Node.js server
 - Connecting to PostgreSQL
 - Building a simple frontend with HTML and JavaScript
 - CRUD operations in the web app
 - Create
 - Read
 - Update
 - Delete

Prerequisites

Tools Needed

1. Node.js
 - a. JavaScript runtime for backend
2. PostgreSQL
 - a. Database to store data
3. HTML/CSS
 - a. For building frontend
4. JavaScript
 - a. For client side scripting

Node.js

- Download from nodejs.org
- Check the installation:
Node -v
Npm -v

Creating the Project

- Create a new folder for the project

```
mkdir test-webapp  
cd test-webapp
```

- Initialize the project

```
npm init -y
```

This command initializes a new node.js project by creating a package.json file. This file manages the project's dependencies, scripts, metadata, and versioning. `-y` accepts all the default options.

- Installing Dependencies

Installing express for backend

```
npm install express pg body-parser cors
```

Express : Framework to create a server

pg : PostgreSQL client for Node.js

body-parser : to parse incoming request

Cors : handling cross-origin requests

Setting up PostgreSQL

- Create a table in postgresQL
- For this lab,
 - I have create a database called `school`
 - Table named `students` with primary key as `ID` and `name`

Backend

- Create a file server.js
- Setting up Node.js server using Express
- Connecting to PostgreSQL database via pg module
- Provide API for CURD operation

1. Importing and Setting up

```
const express = require('express');
const bodyParser = require('body-parser');
const { Pool } = require('pg');
const cors = require('cors');
```

`express` , a web framework for node.js to build web apps.

`Body-parser`, parsing incoming request so that they can be accessed

`Pg`, allowing database to interact

Cors, Cross-Origin Resource Sharing, allowing server to handle request from different domains

2. App initialization

```
const app = express();  
app.use(bodyParser.json());  
app.use(cors());
```

App, instance of express

Line 2, tells app to use body-parser to parse the incoming requests

Line 3, Enables cors

3. Database Connection

```
const pool = new Pool({  
  user: 'postgres',  
  host: 'localhost',  
  database: 'school',  
  password: 'your_password',  
  port: 5432,  
});
```

Creating a pool, to allow multiple connections to database.

User: PostgreSQL username

Host: where the database is hosted (use localhost)

database : name of the database

Password: password for the postgresQL user

Port: PostgreSQL port (default 5432)

4. API Routes (CRUD operations)

- Read all students

```
app.get('/students', async (req, res) => {  
    const result = await pool.query('SELECT  
* FROM students');  
    res.json(result.rows);  
});
```

Get is used to retrieve all students from the students table in the school database.

The pool query executes the SQL query to select all rows from the table.

Then sending the result rows back to the client in JSON format.

- Create a new student

```
app.post('/students', async (req, res) => {  
    const { name } = req.body;  
    await pool.query('INSERT INTO students  
(name) VALUES ($1)', [name]);  
    res.sendStatus(201);  
});
```

This will add a new student to the database

request
Const name = req.body extracts name field from the

Pool query inserts the new students name into the student
table

res.send status sends an HTTP status code back to client
to indicate success

- Update a student by ID

```
app.put('/students/:id', async (req, res)
=> {
    const { id } = req.params;
    const { name } = req.body;
    await pool.query('UPDATE students SET
name = $1 WHERE id = $2', [name, id]);
    res.sendStatus(200);
});
```

This will update the name of an existing student based on
their ID

First extract the parameter from the URL (:id); then
extract the new name from the request body.

- Deleting a student by ID

```
app.delete('/students/:id', async (req,
res) => {
    const { id } = req.params;
    await pool.query('DELETE FROM students
WHERE id = $1', [id]);
    res.sendStatus(200);
});
```

Extracting ID; execute SQL query to delete a student where the ID matches; sends HTTP status code for successful deletion

5. Starting the Server

```
app.listen(3000, () => console.log('Server
running on port 3000'));
```

Starting the server

Listening to port 3000 for incoming HTTP requests

Logs a message to console to indicate the server is running and accessible at <http://localhost:3000>

Frontend

- Create a folder named `public`
- Add a file `index.html` inside `public`

1. HTML Boilerplate

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Web App</title>
```

- **DOCTYPE declaration:** `<!DOCTYPE html>` tells the browser that this is an HTML5 document.
- `<html lang="en">`: Specifies that the language of the document is English.
- `<head>`: Contains metadata about the document, including its character encoding (UTF-8), viewport settings for mobile responsiveness, and the title (Student Web App).
- `<meta charset="UTF-8">`: Ensures that the document supports various characters (like letters, numbers, and symbols).
- `<meta name="viewport" content="width=device-width, initial-scale=1.0">`: Ensures the page is responsive and scales well on different devices.
- `<title>Student Web App</title>`: Sets the title of the webpage, which appears in the browser tab.

2. JavaScript Functions

Inside the `<script>` tag, there are two asynchronous functions: one for **fetching** students and another for **adding** a new student.

a. `fetchStudents()`: Fetches and displays the list of students

```
<script>
  async function fetchStudents() {
    const response = await fetch('/students');
    const students = await response.json();
```

```

const table = document.getElementById('studentTable');
table.innerHTML = "";
students.forEach(student => {
  table.innerHTML += `<tr><td>${student.id}</td><td>${student.name}</td></tr>`;
});
}

```

- `fetchStudents()`:
 - This function is triggered when the page loads (via the `onload="fetchStudents()"` in the body).
 - `fetch('/students')`: Makes an HTTP GET request to the backend `/students` API to retrieve the list of students.
 - `await response.json()`: Waits for the server's response and parses the data (assumed to be JSON) into a JavaScript object.
 - `document.getElementById('studentTable')`: Selects the table body (`<tbody id="studentTable">`) where the students will be displayed.
 - `table.innerHTML = ""`: Clears any existing content in the table.
 - `students.forEach(...)`: Iterates over the list of students and dynamically adds rows to the table for each student, using template literals (``<tr><td>${student.id}</td><td>${student.name}</td></tr>``).

b. `addStudent()`: Adds a new student to the database

```

async function addStudent() {
  const name = document.getElementById('newName').value;
  await fetch('/students', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ name })
  });
  fetchStudents();
}
</script>

```

- `addStudent()`:
 - Triggered when the user clicks the “Add Student” button.
 - `document.getElementById('newName').value`: Retrieves the value from the input field where the user enters the student's name.

- `fetch('/students', { ... })`: Sends a POST request to the `/students` API with the new student's name. The `fetch()` method is configured to:
 - Use the HTTP POST method to add data to the server.
 - Send the content as JSON (Content-Type: 'application/json').
 - Convert the name into a JSON object and send it in the request body.
- `fetchStudents()`: After adding the student, this function is called again to refresh the list of students and display the newly added student.

3. Body Section

```

</head>
<body onload="fetchStudents()">
  <h1>Student List</h1>
  <table border="1">
    <thead><tr><th>ID</th><th>Name</th></tr></thead>
    <tbody id="studentTable"></tbody>
  </table>
  <h2>Add New Student</h2>
  <input type="text" id="newName" placeholder="Student Name">
  <button onclick="addStudent()">Add Student</button>
</body>
</html>

```

- `<body onload="fetchStudents()">`: When the page loads, it automatically calls `fetchStudents()` to display the list of students.
- `<h1>Student List</h1>`: A header for the student list.
- `<table>`:
 - A table is created to display the list of students.
 - `<thead>`: Defines the header of the table with columns for ID and Name.
 - `<tbody id="studentTable">`: The body of the table where student data will be dynamically inserted by the `fetchStudents()` function.
- **Add New Student Form:**
 - `<input type="text" id="newName" placeholder="Student Name">`: A text input field where the user can type the name of the new student.

- `<button onclick="addStudent()">Add Student</button>`: A button that, when clicked, triggers the `addStudent()` function to send the new student's name to the server.

Running the Application

- Start the PostgreSQL service
- Run the node.js server
 - Node server.js
- Open the browser and visit <http://localhost:3000>